# Special Participation E - Prompt for AI Generating Visuals

Rahul Bir

December 2025

## 1 Project Setup Instructions

This section describes how to create a fresh project environment suitable for generating the interactive visualization tools referenced later in this document. The steps below assume that Node.js is installed on the system.

### 1. Install `npm`

If `npm` is not already available on the machine, install Node.js from the official website. The installation includes `npm`, which is required for managing packages and running development scripts.

### 2. Initialize a New Next.js Project

To create a new Next.js App Router project with TypeScript, ESLint, and Tailwind CSS configured automatically, run:

```
npx create-next-app@latest my-project --typescript --eslint
--app
```

This generates a full directory structure for a modern Next.js application.

### 3. Enter the Project Directory

Move into the newly created folder:

```
cd my-project
```

### 4. Install `shadcn/ui`

Once inside the project directory, initialize the `shadcn/ui` component system, which provides a collection of accessible, themeable UI primitives:

```
npx shadcn@latest init
```

This command prepares the project for generating interface components such as cards, buttons, inputs, and layout elements used in educational visualizations.

### 5. Use the AI Prompt to Generate the Visualization

After the environment is set up, use the general-purpose AI visualization prompt provided in the next section. The AI model will generate the necessary files, components, math utilities, and visual elements inside the Next.js project.

### 6. Start the Development Server

To run the application and view the generated visualization in a browser, execute:

```
npm run dev
```

The application will be accessible locally (typically at `http://localhost:3000`), allowing interactive exploration of the visualization.

## 2 General-Purpose AI Visualization Generation Prompt

This is a reusable prompt designed for generating interactive educational visualizations with the assistance of large language models. It supports concepts from deep learning and linear algebra.

### Concept to Visualize

Insert the concept here:

```
>>> INSERT CONCEPT HERE <<<
```

### Project Environment Description

The project in which this visualization will be generated already includes:

- a fully initialized Next.js App Router project (version 13 or 14),
- Tailwind CSS configured and working,
- `shadcn/ui` installed and available for UI components,
- React and TypeScript set up for client-side interactivity,
- optional support for React Three Fiber and `drei` for 3D visualizations,
- an existing file structure inside the `/app` directory following the App Router layout.

The goal of the prompt is not to set up these tools, but to leverage them to generate an interactive, pedagogically meaningful artifact.

## Task Description

The objective is to build an interactive visualization tool that helps a student understand the chosen concept using small, manipulable toy examples (such as 2D or 3D vectors, 2×2 or 3×3 matrices, simple tensors, or minimal update rules). The visualization should make the underlying mathematical structure clear and intuitive.

## Pedagogical Expectations

To support learning through experimentation and direct interaction, the visualization should adhere to the following principles:

- Use small matrices (2×2 or 3×3) so their behavior is easy to inspect.
- Use 2D or 3D vectors when geometric interpretation is helpful.
- Clearly label all intermediate objects produced during the computation.
- Break the concept into discrete, sequential steps that can be visualized independently.
- Provide short explanatory notes alongside or within the interface to guide understanding.

Example patterns include:

- For the SVD: show the action of $V^\top$, then $\Sigma$, and finally $U$ on a vector.
- For Newton–Schulz: display the singular values of a matrix evolving over iterations.
- For attention mechanisms: visualize similarity scores, exponentiation, and normalization.

## User Interface Expectations

The generated visualization should contain:

- a header describing the concept and learning objective,
- an interactive panel for modifying input parameters (e.g., matrix entries, vectors, coefficients),
- a live-updating visualization area that reflects user inputs,
- brief explanatory text or tooltips tied to each visual element,
- a "Reset" control to restore default values for experimentation.

## Code Style and Organization

Because the supporting environment is already configured, the generated code should:

- use components from `shadcn/ui` such as `Card`, `Button`, `Input`, and `Label`,
- follow the Next.js App Router conventions (placing components in the `/app` directory),
- rely on standard React Hooks such as `useState` and `useMemo`,
- include comments that clarify the purpose of each computation and visualization step,
- implement helper functions in a clear and readable fashion.

## Expected Output from the AI Model

When prompted, the AI model should:

1. present a clear, high-level description of the intended visualization and user interaction flow,
2. produce the full implementation for the visualization in a structured and self-contained form,
3. optionally propose extensions or refinements that could enhance the learning experience.

## Tone and Style

All AI-generated text and code should be:

- mathematically accurate,
- conceptually simple and approachable,
- visually minimal and easy to read,
- oriented toward teaching rather than technical optimization,
- explicit about steps, avoiding unnecessary abstraction.

Small toy examples should be prioritized, and all visual or mathematical transformations should be made explicit so that students can connect computation to intuition.

### Metaprompting Requirement

Before producing final code or visual elements, the AI must explain its reasoning step by step. It should describe how it plans to break the concept into visual components and computational stages. This explicit reasoning aligns with the instructional purpose of the tool and supports transparency in the AI-assisted workflow.